

✓ Tutorial 10 – Model Selection

This notebook contains all the sample code and solutions to the model assessment for model selection.

You will learn:

1. The use of validation dataset
2. Linear regression models with regularizations.
3. Post pruning decision tree with complexity penalty
4. Cross-validation

Summary of toolbox:

1. pruning decision trees: cost_complexity_pruning_path https://scikit-learn.org/stable/auto_examples/tree/plot_cost_complexity_pruning.html
2. Support vector machine for classification: from sklearn.svm import SVC <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
3. GridSearch CV: from sklearn.model_selection import GridSearchCV https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
4. Classification report: from sklearn.metrics import classification_report https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

✓ 1. Ridge Regression with pre-determined penalty

reference: <https://www.analyticsvidhya.com/blog/2016/01/ridge-lasso-regression-python-complete-tutorial/>

✓ 1.1 Lets fist generate a dataset from $y=\sin(x)+\text{noise}$ for fitting

Assignment Week 10 – Due Week 11

1. Use the Breast Cancer Wisconsin (Diagnostic) dataset to select the best model among Decision Trees, RandomForest, and SVC that has the minimum testing error (use the default model settings) (50 points).
 2. Use the same dataset and try to use nest Cross-validation to get the optimal hyperparameters for each model, and then compare the model performance (50 points).
 3. (Bonus 1 point). What other BDA process can you use to further improve the performance of the best models in Question 2? For example, feature selection or dimension reduction, imbalanced data sampling, etc., Please evaluate.
1. Use the Breast Cancer Wisconsin (Diagnostic) dataset to select the best model among Decision Trees, RandomForest, and SVC that has the minimum testing error (use the default model settings) (50 points).

```
# Import necessary libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Step 1: Load the dataset
data = load_breast_cancer()
X = data.data # Features
y = data.target # Target (0 = malignant, 1 = benign)

# Step 2: Split into training and testing sets (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # Stratify to maintain class balance
)

# Step 3: Standardize features (required for SVC)
scaler = StandardScaler()
```

```

X_train_scaled = scaler.fit_transform(X_train) # Fit on training data
X_test_scaled = scaler.transform(X_test) # Transform test data (no fitting)

# Step 4: Initialize and train models with default settings
# Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# Random Forest
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

# SVC (uses scaled features)
svc_model = SVC(random_state=42)
svc_model.fit(X_train_scaled, y_train)

# Step 5: Evaluate models on test data
# Decision Tree predictions
dt_pred = dt_model.predict(X_test)
dt_accuracy = accuracy_score(y_test, dt_pred)
dt_error = 1 - dt_accuracy

# Random Forest predictions
rf_pred = rf_model.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)
rf_error = 1 - rf_accuracy

# SVC predictions (uses scaled test data)
svc_pred = svc_model.predict(X_test_scaled)
svc_accuracy = accuracy_score(y_test, svc_pred)
svc_error = 1 - svc_accuracy

# Step 6: Print results
print("Model Performance (Testing Error):")
print(f"Decision Tree: {dt_error:.4f}")
print(f"Random Forest: {rf_error:.4f}")
print(f"SVC: {svc_error:.4f}")

# Identify the best model
best_model = min(
    [("Decision Tree", dt_error), ("Random Forest", rf_error), ("SVC", svc_error)],
    key=lambda x: x[1]
)
print(f"\nBest Model (Minimum Testing Error): {best_model[0]} (Error: {best_model[1]:.4f})")

```

Model Performance (Testing Error):

Decision Tree: 0.0877

Random Forest: 0.0439

SVC: 0.0175

Best Model (Minimum Testing Error): SVC (Error: 0.0175)

2. Use the same dataset and try to use nest Cross-validation to get the optimal hyperparameters for each model, and then compare the model performance (50 points).

```

# Import necessary libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
import numpy as np

# Step 1: Load the dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Step 2: Define hyperparameter grids for each model
# Decision Tree hyperparameters
dt_param_grid = {
    'max_depth': [None, 5, 10, 15],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Random Forest hyperparameters
rf_param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 5, 10],

```

```

    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# SVC hyperparameters (uses scaled features)
svc_param_grid = {
    'svc__C': [0.1, 1, 10, 100],
    'svc__gamma': ['scale', 'auto', 0.001, 0.01, 0.1, 1],
    'svc__kernel': ['rbf', 'linear']
}

# Step 3: Set up nested cross-validation
# Outer CV: 5-fold stratified split
outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize lists to store results
dt_scores = []
rf_scores = []
svc_scores = []

# Step 4: Perform nested CV for each model
print("Performing nested cross-validation...")

# Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_grid_search = GridSearchCV(
    estimator=dt_model,
    param_grid=dt_param_grid,
    cv=3, # Inner CV: 3-fold
    scoring='accuracy',
    n_jobs=-1
)

for fold, (train_idx, test_idx) in enumerate(outer_cv.split(X, y)):
    print(f"\n--- Outer Fold {fold+1}/5 ---")
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    # Tune hyperparameters on inner CV
    print("Tuning Decision Tree...")
    dt_grid_search.fit(X_train, y_train)

    # Evaluate best model on outer test fold
    best_dt = dt_grid_search.best_estimator_
    dt_score = best_dt.score(X_test, y_test)
    dt_scores.append(dt_score)
    print(f"Decision Tree Best Params: {dt_grid_search.best_params_}")
    print(f"Decision Tree Outer Fold Score: {dt_score:.4f}")

# Random Forest
rf_model = RandomForestClassifier(random_state=42)
rf_grid_search = GridSearchCV(
    estimator=rf_model,
    param_grid=rf_param_grid,
    cv=3,
    scoring='accuracy',
    n_jobs=-1
)

for fold, (train_idx, test_idx) in enumerate(outer_cv.split(X, y)):
    print(f"\n--- Outer Fold {fold+1}/5 ---")
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    print("Tuning Random Forest...")
    rf_grid_search.fit(X_train, y_train)
    best_rf = rf_grid_search.best_estimator_
    rf_score = best_rf.score(X_test, y_test)
    rf_scores.append(rf_score)
    print(f"Random Forest Best Params: {rf_grid_search.best_params_}")
    print(f"Random Forest Outer Fold Score: {rf_score:.4f}")

# SVC (uses StandardScaler in a pipeline)
svc_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(random_state=42))
])

svc_grid_search = GridSearchCV(
    estimator=svc_pipeline,
    param_grid=svc_param_grid,
    cv=3,
    scoring='accuracy'
)

```

```

        scoring= accuracy ,
        n_jobs=-1
    )

for fold, (train_idx, test_idx) in enumerate(outer_cv.split(X, y)):
    print(f"\n--- Outer Fold {fold+1}/5 ---")
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    print("Tuning SVC...")
    svc_grid_search.fit(X_train, y_train)
    best_svc = svc_grid_search.best_estimator_
    svc_score = best_svc.score(X_test, y_test)
    svc_scores.append(svc_score)
    print(f"SVC Best Params: {svc_grid_search.best_params_}")
    print(f"SVC Outer Fold Score: {svc_score:.4f}")

# Step 5: Calculate and print results
dt_mean_accuracy = np.mean(dt_scores)
dt_mean_error = 1 - dt_mean_accuracy

rf_mean_accuracy = np.mean(rf_scores)
rf_mean_error = 1 - rf_mean_accuracy

svc_mean_accuracy = np.mean(svc_scores)
svc_mean_error = 1 - svc_mean_accuracy

print("\n--- Final Results ---")
print("Model Performance (Mean Testing Error \u00b1 Std Dev):")
print(f"Decision Tree: {dt_mean_error:.4f} \u00b1 {np.std(dt_scores):.4f}")
print(f"Random Forest: {rf_mean_error:.4f} \u00b1 {np.std(rf_scores):.4f}")
print(f"SVC: {svc_mean_error:.4f} \u00b1 {np.std(svc_scores):.4f}")

# Identify the best model
best_model = min(
    [("Decision Tree", dt_mean_error), ("Random Forest", rf_mean_error), ("SVC", svc_mean_error)],
    key=lambda x: x[1]
)
print(f"\nBest Model (Minimum Mean Testing Error): {best_model[0]} (Error: {best_model[1]:.4f})")

```

```

--- Outer Fold 1/5 ---
Tuning Random Forest...
Random Forest Best Params: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 200}
Random Forest Outer Fold Score: 0.9649

--- Outer Fold 2/5 ---
Tuning Random Forest...
Random Forest Best Params: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 100}
Random Forest Outer Fold Score: 0.9386

--- Outer Fold 3/5 ---
Tuning Random Forest...
Random Forest Best Params: {'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Random Forest Outer Fold Score: 0.9474

--- Outer Fold 4/5 ---
Tuning Random Forest...
Random Forest Best Params: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 50}
Random Forest Outer Fold Score: 0.9561

```

--- Outer Fold 5/5 ---

Tuning SVC...

SVC Best Params: {'svc__C': 10, 'svc__gamma': 0.01, 'svc__kernel': 'rbf'}

SVC Outer Fold Score: 0.9823

--- Final Results ---

Model Performance (Mean Testing Error $\pm$ Std Dev):

Decision Tree: 0.0914 $\pm$ 0.0239

Random Forest: 0.0439 $\pm$ 0.0123

SVC: 0.0228 $\pm$ 0.0189

Best Model (Minimum Mean Testing Error): SVC (Error: 0.0228)